ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #9

SportsCraft v0.2.0

SportsCraft is a local spectator + personal-trainer sports studio. It covers



v0.2.0	4 Crates	3887	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\sportscraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

SportsCraft is a local spectator + personal-trainer sports studio. It covers European soccer as a spectator (demo tables, fixtures, match predictor) and works as a personal trainer in the gym or at home (exercise library, weekly plans, coached single workouts) — on top of the Banister fitness/fatigue/form lab, race pacing guidance, and IMU gait kinematics. Everything runs on your machine: no account, no cloud, no network port.

What works

- **Spectator soccer (European, demo)** — five leagues (Premier League, La Liga, Bundesliga, Serie A, Ligue 1) with standings, upcoming fixtures, and an Elo-style match predictor. All output says `source: "demo"`: no live feed.
- **Personal trainer (gym or home, demo)** — 23-exercise catalog filtered by location, weekly plans for 2-5 days across strength / hypertrophy / conditioning / general goals and beginner / intermediate / advanced levels, and single coached workouts (warm-up, main, cooldown, TRIMP estimate, cues). Guidance is general fitness information, not medical advice.
- **Banister impulse response** — CTL (fitness, 42 d), ATL (fatigue, 7 d) and TSB (form = CTL - ATL) folded from a daily TRIMP series, all in TRIMP points.
- **Race pacing prediction** — finish time and min/km pace for 800 m to 100 km from current chronic fitness, with the race-ready TSB window (`+5 .. +18`).
- **Gait kinematics** — cadence, vertical oscillation, ground contact time, leg spring stiffness and asymmetry. **Demo fixture**: no sensor or file importer is attached yet, and the output says `source: "demo"`.
- **MCP server over stdio** — eleven tools, no TCP port: `status`, `athlete.profile`, `athlete.predict`, `gait.analyze`, `soccer.leagues`, `soccer.standings`, `soccer.fixtures`, `soccer.predict`, `trainer.catalog`, `trainer.plan`, `trainer.session`.
- **Desktop studio** (`eframe` / `egui`) — **Spectator** section (league tables, fixtures, predictor) and **Personal** section (**Trainer** for gym/home plans and workouts, **Lab** for Banister, gait, pacing simulator), theme toggle, and a locale selector for EN / DE / FR / ES / JA / KO / ZH.

What is not implemented

Be aware before installing:

- No FIT/GPX/TCX import. The training history is a deterministic demo block

(`history_source: "demo"`).

- No persistence: sessions are not stored, and there is no workout queue.
- No PDF export. The File menu exports JSON and copies JSON to the clipboard.
- No authentication. There is no account or password.

Workspace

Crate	Role
`crates/sportscraft-kinematics`	Banister solver, gait model (no CLI/MCP/UI deps)
`crates/sportscraft-core`	Pacing prediction, soccer + trainer engines, i18n, `Command::dispatch` bus
`crates/sportscraft-mcp`	MCP `2024-11-05` stdio server (`sportscraft-mcp.exe`)
`crates/sportscraft-app`	Desktop studio (`sportscraft.exe`) and CLI (`sportscraft-cli.exe`)

The desktop app calls core in-process. There is no sidecar, no HTTP server, and no webapp.

Build and run

Requires Rust (stable) only. No Python, Node, or `uv`.

```
# Full gate set: clippy, fmt, tests, MCP stdio probe, app smoke
just ci

# Desktop app
cargo run -p sportscraft-app --bin sportscraft

# CLI
cargo run -p sportscraft-app --bin sportscraft-cli -- status
cargo run -p sportscraft-app --bin sportscraft-cli -- profile ATHLETE-PRO
cargo run -p sportscraft-app --bin sportscraft-cli -- predict ATHLETE-PRO 42195
cargo run -p sportscraft-app --bin sportscraft-cli -- soccer-leagues
cargo run -p sportscraft-app --bin sportscraft-cli -- soccer-standings ENG1
cargo run -p sportscraft-app --bin sportscraft-cli -- trainer-catalog home

# MCP server (stdio; logs on stderr, JSON-RPC on stdout)
cargo run -p sportscraft-mcp
```

`start.ps1` wraps the desktop build/run; `start.bat` is the double-click shim.

MCP client configuration

```
{
  "mcpServers": {
    "sportscraft": {
      "command": "D:\\Dev\\repos\\sportscraft\\target\\release\\sportscraft-mcp.exe",
      "args": []
    }
  }
}
```

See `docs/TOOLS.md` for the tool schemas and

`docs/DEVELOPMENT.md` for the release convention.

Notes on the model

TSB is `CTL − ATL` in TRIMP points. Both impulses use weight 1.0 so the race-ready window (`+5 .. +18`) is meaningful; a 21-day taper from a hard block reaches it. With no taper, a hard 14-day demo block sits around `−53`, and the UI shows the target range so the gap is visible rather than hidden.

Status — does today / next week / non-goals

Does today: Spectator soccer (5 European leagues, tables, fixtures, predictor) + Personal trainer (gym/home catalog, weekly plans, coached workouts) + Banister CTL/ATL/TSB lab + race pacing (800 m–100 km) + IMU gait; all demo output labeled `source: demo`; trainer carries the not-medical-advice line; NSIS installer posture; 30 test fns.

Next week: golden Banister fixtures (known TRIMP → exact CTL/ATL); CSV importer decision + source labeling; banister-sim recipe.

Non-goals: not medical advice (pacing + trainer guidance carry the line); no live scores feed, no wearable auto-sync yet.

Lineage — the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft — "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.

Complete MCP Tool & CLI Verb Reference

SportsCraft exposes the same capability surface three ways: the MCP stdio server, the CLI, and the desktop UI. All of them call `sportscraft_core::Command::dispatch`.

MCP server (`sportscraft-mcp.exe`)

Transport: MCP `2024-11-05` JSON-RPC over **stdio**. No TCP port. Logs go to stderr; stdout carries only JSON-RPC frames.

`status`

No arguments. Engine version, Banister time constants, the race-ready TSB window, the command list, and the demo-data marker.

```
{"status":"ready","engine":"sportscraft","version":"0.2.0",
 "fitness_tau_days":42.0,"fatigue_tau_days":7.0,
 "optimal_tsb_window":[5.0,18.0],"history_source":"demo"}
```

`athlete.profile`

Argument	Type	Required	Default
`athlete`	string	no	`ATHLETE-ELITE-1`

Returns the Banister profile: CTL (`current_fitness`), ATL (`current_fatigue`), TSB (`form_tsb`), the time constants and impulse weights, and `history_source: "demo"`.

`athlete.predict`

Argument	Type	Required
`distance_m`	number (`> 0`)	yes
`athlete`	string	no (default `ATHLETE-ELITE-1`)

Returns `target_distance_m`, `predicted_time_seconds`, `recommended_pace_min_km`, and `optimal_form_tsb_window`. A zero, negative, or non-finite distance returns `success: false` with an error string.

`gait.analyze`

No arguments. Cadence, vertical oscillation, ground contact time, leg spring stiffness, asymmetry, and `source: "demo"` (the values are a fixture; no sensor is attached).

`soccer.leagues`

No arguments. The five demo European leagues: Premier League (`ENG1`), La Liga (`ESP1`), Bundesliga (`GER1`), Serie A (`ITA1`), Ligue 1 (`FRA1`).

Returns `source: "demo"`.

`soccer.standings`

Argument	Type	Required
`league`	string (`ENG1` , `ESP1` , `GER1` , `ITA1` , `FRA1`)	yes

Demo table: position, played/won/drawn/lost, goals, points (`3*W+D`), last-five form. Unknown codes return `success: false` . Returns `source: "demo"` .

`soccer.fixtures`

Argument	Type	Required
`league`	string (same codes)	yes

Demo next-matchday fixtures with kickoff labels and venues. Unknown codes return `success: false` . Returns `source: "demo"` .

`soccer.predict`

Argument	Type	Required
`home`	string (team name, e.g. `Man City`)	yes
`away`	string (team name, e.g. `Arsenal`)	yes

Elo-style demo predictor from the bundled ratings plus home edge: home/draw/away percentages, expected goals, and a pick. Unknown team names return `success: false` . Returns `source: "demo"` .

`trainer.catalog`

Argument	Type	Required
`location`	string (`gym` or `home`)	yes

Exercise library filtered by location (`gym` sees `gym` + `both` , `home` sees `home` + `both`): muscle, equipment, sets/reps/rest, coaching cues. Returns `source: "demo"` plus the general-fitness disclaimer.

`trainer.plan`

Argument	Type	Required
`goal`	string (`strength` , `hypertrophy` , `conditioning` , `general`)	yes
`location`	string (`gym` or `home`)	yes
`days_per_week`	integer (2-5)	yes
`level`	string (`beginner` , `intermediate` , `advanced`)	yes

Weekly plan: one focused day per training day with exercises, estimated minutes and TRIMP. Out-of-range day counts return `success: false`. Returns `source: "demo"` plus the general-fitness disclaimer (not medical advice).

`trainer.session`

Argument	Type	Required
`goal`	string (same four)	yes
`location`	string (`gym` or `home`)	yes
`level`	string (same three)	yes
`minutes`	integer (15-120)	yes

One coached workout: warm-up, main sets, cooldown, TRIMP estimate, coaching cues. Out-of-range minutes return `success: false`. Returns `source: "demo"` plus the general-fitness disclaimer (not medical advice).

Underscore aliases (`athlete_profile` , `athlete_predict` , `gait_analyze` , `soccer_leagues` , `soccer_standings` , `soccer_fixtures` , `soccer_predict` , `trainer_catalog` , `trainer_plan` , `trainer_session`) route to the same handlers as the dotted names.

CLI (`sportscraft-cli.exe`)

```
sportscraft-cli status
sportscraft-cli profile  <athlete>
sportscraft-cli predict  <athlete> <distance_m>
sportscraft-cli gait
sportscraft-cli soccer-leagues
sportscraft-cli soccer-standings  <league>
sportscraft-cli soccer-fixtures   <league>
sportscraft-cli soccer-predict    <home> <away>
sportscraft-cli trainer-catalog   <location>
sportscraft-cli trainer-plan      <goal> <location> <days_per_week> <level>
sportscraft-cli trainer-session   <goal> <location> <level> <minutes>
```

Each command prints the same JSON payload as the MCP tool (pretty-printed).

Desktop UI

- **Spectator** — league selector, standings table, fixtures, match predictor

(all demo data, labeled `source: demo`).

- **Personal** → **Trainer** — goal / gym-or-home / level / days / minutes controls,

weekly plan builder, single-workout builder, exercise library. Guidance is

general fitness information, not medical advice.

- **Personal** → **Lab** — Banister panel, gait panel, pacing simulator.
- **File** — export the pacing plan as JSON (real file, real path reported) or

copy it to the clipboard.

- **Distances** — 5K / 10K / half / marathon / 100K presets.
- **View** — light/dark theme.
- **Help** — About (version, transport, what is and is not implemented).
- **Locale selector** — EN / DE / FR / ES / JA / KO / ZH; every UI string is

localized through `sportscraft_core::i18n`.